

BookTrendsRecSys — Technical Report

Team BookTrends

October 16, 2025

Abstract

We present **BookTrendsRecSys**, a compact recommendation system that produces a Top-10 list of books tailored to a reader. The pipeline covers *data collection*, *data preparation*, *exploratory data analysis (EDA)*, an *implicit-feedback ALS* model, *offline evaluation* with order-aware ranking metrics (Precision@10, Recall@10, NDCG@10), and a *Streamlit* app for serving results and dataset insights. We include a lightweight fairness snapshot across regions/genres (smaller gap $\Rightarrow$ more similar quality). Artifacts (metrics, parity tables, EDA images) are kept in versioned folders for reproducibility. This report documents design choices, limitations, and how to run the full system end-to-end.

1 Introduction

Readers and librarians face choice overload. Instead of predicting exact star ratings, we aim to surface a short, high-quality list. We therefore optimize and report **ranking** metrics directly connected to user experience: *Precision@10* (hit-rate in the 10 shown), *Recall@10* (coverage of relevant items in 10), and *NDCG@10* (order-aware list quality). Model hyperparameters are tuned on validation NDCG@10, then frozen for final testing and app serving. Seed is fixed to 42.

2 Data Collection

Sources & Scripts. The repository includes a data-collection area and notes under `data_collection_scripts`. Raw CSVs live in `data/raw/` (`books.csv`, `ratings.csv`). A typical ordered pipeline is: seed books $\rightarrow$ crawl book reviews $\rightarrow$ crawl user histories $\rightarrow$ aggregate (`user`, `book`, `rating`, `timestamp`) $\rightarrow$ QA coverage stats.

3 Dataset & EDA

Structure. Clean separation of inputs/processed/figures:

- `data/raw/`: `books.csv`, `ratings.csv`
- `data/processed/`: `train/val/test.parquet`, lookups (`books.csv`, `users.csv`), summaries (`eda_summary.json`)
- `figs/eda/`: exported charts used in app/report

4 Methods: Implicit ALS

We use the implicit-feedback **Alternating Least Squares (ALS)** formulation (Hu, Koren, Volinsky 2008). For observed interactions r_{ui} , we form binary preferences $p_{ui} \in \{0, 1\}$ and

confidence $c_{ui} = 1 + \alpha r_{ui}$. We factorize into X (users) and Y (items) by minimizing:

$$\min_{X,Y} \sum_{u,i} c_{ui} (p_{ui} - x_u^\top y_i)^2 + \lambda (\|x_u\|_2^2 + \|y_i\|_2^2),$$

via alternating least-squares updates. Hyperparameters (rank, λ , α , iterations) are selected by validation NDCG@10.

5 Evaluation: Metrics & Results

Let G_u be the held-out relevant set for user u , and $R_u = [i_1, \dots, i_K]$ the top- K ranking ($K=10$). We compute:

$$\begin{aligned} \text{P@K} &= \frac{1}{|U|} \sum_u \frac{|R_u \cap G_u|}{K}, & \text{R@K} &= \frac{1}{|U|} \sum_u \frac{|R_u \cap G_u|}{|G_u|}, \\ \text{DCG@K}(u) &= \sum_{j=1}^K \frac{\mathbf{1}[i_j \in G_u]}{\log_2(j+1)}, & \text{NDCG@K} &= \frac{1}{|U|} \sum_u \frac{\text{DCG@K}(u)}{\text{IDCG@K}(u)}. \end{aligned}$$

Binary relevance is used; users with empty G_u are skipped in averages.

6 Fairness Snapshot

We compute the same metric (typically NDCG@10) by group (e.g., region or genre) and report the *gap*:

$$\text{gap} = \max_g \text{metric}(g) - \min_g \text{metric}(g).$$

Smaller gaps imply more similar quality across groups. See `metrics/parity.csv`.

7 Application & Serving

Interface. The Streamlit app (`app/app.py`) provides:

- **Recommendations:** choose a user/genre; see a short Top-10 list.
- **Insights:** friendly KPI cards and EDA figures (ratings, languages, long tail, etc.).

Run locally.

1. `pip install -r requirements.txt`
2. `make data && make train && make eval && make parity`
3. `streamlit run app/app.py`

8 Reproducibility

Seed fixed at 42; clean train/val/test protocol. Artifacts live under `models/`, `metrics/`, `figs/eda/`, `data/processed/`. Use `latexmk -pdf report.tex` from the `report/` folder to compile this document.

9 Limitations & Future Work

Implicit data favors popularity; cold start remains challenging. Offline metrics are proxies; consider online tests. Future: content-aware models, exposure/coverage parity, and improved fallbacks for sparse slices.